Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado en Ingeniería Informática

Memoria final TFG

Desarrollo de una Aplicación Móvil para la Alerta de Caducidad de Medicamentos

Autor: Christian Toral Ontoria
Tutor(a): Alejandro Rodríguez González

Madrid, marzo de 2023

Esta memoria final se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su corrección.

Memoria final
Grado en Ingeniería Informática
Título: Desarrollo de una Aplicación Móvil para la Alerta de Caducidad de Medicamentos
Marzo de 2023

Autor: Christian Toral Ontoria

Tutor: Alejandro Rodríguez González
Lenguajes y Sistemas Informáticos e Ingeniería de Software
ETSI Informáticos
Universidad Politécnica de Madrid

Resumen

Este documento presenta una memoria sobre el proyecto de desarrollo de una aplicación móvil que permite la gestión y programación de alertas de caducidad de medicamentos mediante la toma de fotografías de cajas de medicamentos para identificar el nombre y la fecha de caducidad de los mismos. La información se guarda en una base de datos en línea, lo que permite compartir la lista de medicamentos con otros usuarios y recibir dichas alertas de caducidad en todos los dispositivos vinculados con una cuenta.

El capítulo 1 presenta una introducción al proyecto y los objetivos del mismo, mientras que el capítulo 2 se enfoca en el estudio y comparación de aplicaciones similares. El capítulo 3 describe el desarrollo de la aplicación, incluyendo el entorno utilizado, las bases de datos empleadas, el reconocedor de imágenes y las clases que componen el proyecto. El capítulo 4 presenta las pruebas de funcionamiento de la aplicación, en las que se evaluó la usabilidad, el rendimiento, la integración y el reconocimiento de imágenes. El capítulo 5 se enfoca en posibles planes futuros para la aplicación, como la integración con asistentes virtuales, mejoras en el reconocimiento de imágenes y la integración con farmacias. Finalmente, el capítulo 6 presenta un informe de impacto en la sociedad, ya que la gestión inadecuada de medicamentos puede tener consecuencias graves para la salud. La aplicación propuesta permitiría a los usuarios tener un control más efectivo de sus medicamentos y la fecha de caducidad de los mismos, lo que podría mejorar la calidad de vida de muchos usuarios.

Abstract

This document presents a project report on the development of a mobile application that allows the management and scheduling of medication expiration alerts by taking pictures of medication boxes to identify the name and expiration date of the medication. The information is stored in an online database, allowing the medication list to be shared with other users and to receive such expiration alerts on all devices linked to an account.

Chapter 1 presents an introduction to the project and the project objectives, while Chapter 2 focuses on the study and comparison of similar applications. Chapter 3 describes the development of the application, including the environment used, the databases used, the image recognizer and the classes that make up the project. Chapter 4 presents the functional tests of the application, in which usability, performance, integration and image recognition were evaluated. Chapter 5 focuses on possible future plans for the application, such as integration with virtual assistants, improvements in image recognition and integration with pharmacies. Finally, Chapter 6 presents a societal impact report, as inadequate medication management can have serious health consequences. The proposed application would allow users to have a more effective control of their medications and their expiration date, which could improve the quality of life of many users.

Tabla de contenidos

1 Introducción 1
1.1 Contexto y motivación 1
1.2 Problema y objetivos 1
2 Metodología de Trabajo 2
2.1 Descripción de la metodología utilizada en el proyecto 2
3 Estudio y comparación de aplicaciones similares 3
3.1 Aplicaciones móviles existentes 3
3.2 Características principales y limitaciones 3
3.3 Métodos y técnicas utilizados 3
3.4 Conclusiones del estudio 4
4 Fase de Análisis 5
4.1 Requerimientos Funcionales 5
4.2 Requerimientos no Funcionales 6
4.3 Requisitos Funcionales 6
4.4 Requisitos no Funcionales 9
4.5 Características de los Usuarios 11
5 Fase de Diseño 12
5.1 Diseño de la Arquitectura 12
5.2 Diseño de la Interfaz 14
5.3 Evaluación del Diseño 14
5.4 Herramientas 14
5.4.1 Entorno utilizado: Android Studio 14
5.4.1.1 Resumen 14
5.4.1.2 Implementación del proyecto 15
5.4.2 Bases de datos empleadas 15
5.4.2.1 Base de datos Firebase Firestore 15
5.4.2.2 Base de datos SQL 17
5.4.3 Reconocedor de imágenes 17
5.4.3.1 Procedimiento del reconocedor de imágenes 17
5.4.3.2 Limitaciones del reconocedor de imágenes 18
6 Fase de Desarrollo 19
6.1 Breve explicación de las clases que componen el proyecto. 19
6.1.1 MainActivity 19
6.1.2 LoginActivity 20
6.1.3 RegisterActivity 21
6.1.4 PasswordRecoveryActivity y PssdRecoveryAnimationActivity 22
6.1.5 HomeMainPageActivity 22
6.1.6 MainPageActivity 24

6.1.7 MedicamentListActivity 24
6.1.8 MedPageActivity 25
6.1.9 ListsFragment 26
6.1.10 MedListFragment 26
6.1.11 ShareListsActivity 27
6.1.12 ShareFragment 27
6.1.13 ExpiredMedsFragment 28
6.1.14 AboutFragment 30
6.1.15 ProfileFragment 31
6.1.16 FirebaseMessagingService_medCheck 31
6.1.17 NotificationReceiver_medCheck 32
6.1.18 NotificationSender_medCheck 32
7 Pruebas de funcionamiento 33
7.1 Pruebas de usabilidad 33
7.1.1 Prueba 1: Registro de usuarios 33
7.1.2 Prueba 2: Inicio de sesión y verificación de la cuenta 34
7.1.3 Prueba 3: Añadir, buscar, modificar y eliminar listas 34
7.1.3.1 Comprobaciones extra derivadas de la prueba 3 35
7.1.4 Prueba 4: Añadir medicamento 35
7.1.4.1 Comprobaciones extra derivadas de la prueba 4 35
7.1.5 Prueba 5: Ver información de un medicamento 36
7.1.6 Prueba 6: Compartir lista de medicamentos 36
7.1.7 Prueba 7: Gestión de errores 37
7.2 Pruebas de rendimiento 37
7.2.1 Prueba 1: Prueba de carga 37
7.3 Pruebas de integración 38
7.3.1 Prueba 1: Sincronización con la nube 38
7.3.2 Prueba 2: Prueba de integración 38
7.4 Pruebas del reconocedor de imágenes 39
7.4.1 Prueba 1: Fotografías borrosas 39
7.4.2 Prueba 2: Fotografías en otro idioma 39
7.4.3 Prueba 3: Fotografías claras 40
7.4.4 Prueba 4: Fotografías parcialmente cubiertas 40
7.5 Análisis de los resultados 40
8 Posibles planes futuros para la aplicación 41
8.1 Integración con asistentes virtuales 41
8.2 Mejoras en el reconocimiento de imágenes 41
8.3 Integración con farmacias 41
9 Informe de impacto en la sociedad 42
10 Bibliografía 43

1 Introducción

El TFG tiene como objetivo desarrollar una aplicación móvil (Android) que permita realizar fotografías de cajas de medicamentos e identificar de forma automática dos valores: por una parte, el nombre del medicamento, y por otra, la fecha de caducidad, mediante dos tomas de fotografías.

La información será guardada en una base de datos online, lo que permitirá que distintos usuarios puedan compartir la misma lista de medicamentos (familiares, amigos, etc.). Además, se creará una "alerta" una vez el medicamento haya caducado, para informar al usuario de esta caducidad.

La aplicación, por lo tanto, permitirá al usuario tener un listado de los medicamentos que tiene en casa, con sus fechas de caducidad, y alarmas relativas a estas fechas.

1.1 Contexto y motivación

La toma de medicamentos es una práctica común en la sociedad actual, especialmente en la tercera edad. Sin embargo, el uso inadecuado de medicamentos puede tener consecuencias graves para la salud, como efectos secundarios no deseados, interacciones medicamentosas y errores en la dosificación. Además, muchos medicamentos tienen una fecha de caducidad, después de la cual pueden perder su eficacia o incluso ser peligrosos para la salud. Por esta razón, es importante que los usuarios conozcan la fecha de caducidad de sus medicamentos y los reemplacen a tiempo.

La gestión de los medicamentos y su fecha de caducidad puede resultar complicada para algunos usuarios, especialmente aquellos que toman múltiples medicamentos o que tienen problemas de memoria. En este sentido, una aplicación móvil que permita la gestión de los medicamentos y la alerta de caducidad puede ser de gran utilidad para los usuarios. Además, el proyecto no trata solo de una gestión orientada a la fecha de caducidad, sino que también permite a los usuarios verificar qué medicamentos tienen en casa, evitando así que compren medicamentos que ya tienen, algo que suele ocurrir con frecuencia con fármacos de uso poco común.

1.2 Problema y objetivos

El problema que se aborda en este proyecto es la gestión de los medicamentos y la alerta de caducidad. Se pretende desarrollar una aplicación móvil que permita a los usuarios gestionar sus medicamentos y recibir una alerta cuando alguno de sus medicamentos está a punto de caducar.

Los objetivos específicos del proyecto son:

- Desarrollar una aplicación móvil para la alerta de caducidad de medicamentos.
- Permitir a los usuarios gestionar sus medicamentos y recibir una alerta cuando alguno de sus medicamentos está a punto de caducar.
- Proporcionar información detallada sobre los medicamentos y sus efectos secundarios.

2 Metodología de Trabajo

A continuación, se describirá la metodología utilizada para el desarrollo del proyecto, en el que se incluirá toda la información relacionada con los métodos de análisis, diseño, desarrollo y pruebas.

2.1 Descripción de la metodología utilizada en el proyecto

Para desarrollar este proyecto, se optó por utilizar una metodología de desarrollo ágil, basada en sprints de trabajo. Además, se decidió dividir el trabajo en cuatro fases: fase de análisis, diseño, desarrollo y pruebas de funcionamiento.

El propósito de la primera fase es definir y comprender el problema a resolver, sus objetivos y los requerimientos del usuario, para así poder diseñar una solución adecuada. Para ello, se realizó un análisis riguroso de los requisitos del proyecto y se definieron meticulosamente los objetivos y metas del proyecto.

La segunda fase, la de diseño, tiene como objetivo planificar cómo será la aplicación, incluyendo la interfaz de usuario, la funcionalidad y la experiencia del usuario.

La fase de desarrollo es la etapa en la que se implementa el diseño de la aplicación y se construye la funcionalidad. En este proyecto, la fase de desarrollo se ha llevado a cabo utilizando Android Studio.

Por último, La fase de pruebas es una etapa importante en el desarrollo de software, ya que tiene como objetivo detectar errores y asegurar que la aplicación funciona correctamente. Durante esta fase se llevaron a cabo diferentes tipos de pruebas exhaustivas, como pruebas de rendimiento, pruebas de integración y pruebas del reconocedor, entre otras. Estas pruebas permiten detectar errores y mejorar la calidad de la aplicación.

Cada fase contó con revisiones al final de la misma, con el fin de asegurar que los objetivos y las metas de cada fase se cumplieran satisfactoriamente antes de continuar con la siguiente.

Se espera que el proyecto tenga una duración de 12 semanas, comenzando el 20 de febrero 2023 y finalizando el 31 de mayo de 2023.

3 Estudio y comparación de aplicaciones similares

En este capítulo se revisan otros trabajos previos relacionados con la alerta de caducidad de medicamentos y su gestión. Se describen las aplicaciones móviles existentes que abordan este problema, así como sus características principales y limitaciones. También se discuten los métodos y técnicas utilizados en estos trabajos para abordar el problema de la alerta de caducidad de medicamentos.

3.1 Aplicaciones móviles existentes

Existen otras aplicaciones móviles que ayudan a gestionar los medicamentos y a recordar la toma de los mismos, tales como:

- CareZone: esta aplicación móvil permite llevar un registro de los medicamentos y recordatorios de toma. También ofrece información detallada sobre los medicamentos y la posibilidad de compartir la información con los cuidadores o familiares. La aplicación es gratuita [1].

- DailyMed: esta aplicación móvil permite también llevar un registro de medicamentos, recordatorios de toma y proporciona información detallada sobre cada medicamento. La aplicación es gratuita [2].

3.2 Características principales y limitaciones

Aunque estas aplicaciones móviles tienen como objetivo abordar el problema de la alerta de caducidad de medicamentos, presentan algunas limitaciones. En general, estas aplicaciones móviles se centran en la gestión de medicamentos y la programación de recordatorios, pero no ofrecen una solución completa al problema de la alerta de caducidad de medicamentos.

Además, estas aplicaciones móviles suelen depender de la entrada manual de datos por parte de los usuarios, lo que puede ser una barrera para algunos usuarios. Por ejemplo, algunas personas mayores pueden tener dificultades para introducir información en una aplicación móvil debido a problemas de destreza o falta de experiencia con la tecnología.

3.3 Métodos y técnicas utilizados

Para abordar el problema de la alerta de caducidad de medicamentos, se han utilizado diversas técnicas y métodos en trabajos previos. Entre estos métodos, se pueden destacar los siguientes:

- Sistemas de alerta: algunos trabajos previos han utilizado sistemas de alerta para notificar a los usuarios cuando sus medicamentos están a punto de caducar. Estos sistemas suelen estar integrados en una aplicación móvil o en un dispositivo de almacenamiento de medicamentos.

- Reconocimiento de voz: algunos trabajos previos han utilizado el reconocimiento de voz para permitir a los usuarios introducir información en una aplicación móvil de manera más eficaz. Esta técnica puede ser útil para usuarios con problemas de destreza o falta de experiencia con la tecnología.

- Análisis de datos: algunos trabajos previos han utilizado técnicas de análisis de datos para identificar patrones en los hábitos de toma de medicamentos de los usuarios. Estos patrones pueden ser útiles para personalizar la gestión de medicamentos y mejorar la eficacia de las alertas de caducidad.

3.4 Conclusiones del estudio

A partir de esta revisión, se ha identificado la necesidad de desarrollar una aplicación móvil que ofrezca una solución completa al problema de la alerta de caducidad de medicamentos, y que esté dirigida a grupos de usuarios con necesidades especiales.

Por tanto, la aplicación que estoy desarrollando tiene varias ventajas respecto a los trabajos previos que se han abordado en la misma línea. En primer lugar, la aplicación utiliza la plataforma Firebase (FirebaseAuth y FirebaseFirestore), lo que permite una gestión más eficiente y segura de la base de datos y de los usuarios. Además, la aplicación cuenta con una interfaz de usuario atractiva y fácil de usar, lo que mejora la experiencia de usuario y la hace más atractiva y usable [5].

En cuanto a los métodos y técnicas utilizados, la aplicación combina diferentes técnicas para abordar el problema de la alerta de caducidad de medicamentos. Por un lado, la aplicación utiliza sistemas de alerta para notificar a los usuarios cuando sus medicamentos están a punto de caducar. Por otro lado, la aplicación utiliza técnicas reconocimiento de imágenes para identificar los medicamentos de los usuarios, lo que puede mejorar la eficacia de introducción de los datos, las alertas de caducidad y la gestión de medicamentos.

En el siguiente capítulo se describirá el proceso de diseño de la aplicación, incluyendo las especificaciones técnicas y las características y funcionalidades principales de la aplicación.

4 Fase de Análisis

En este apartado se expone la fase de análisis, cuyo objetivo principal es el de recopilar la información necesaria para entender las necesidades del usuario y los requisitos funcionales y no funcionales del sistema. Para ello se tomaron en cuenta los puntos de vista del tutor y del autor de este documento, se analizaron los requisitos del sistema y se estudiaron las características de los usuarios para determinar las especificaciones y funcionalidades necesarias del sistema.

4.1 Requerimientos Funcionales

- La aplicación debe permitir a los usuarios crear una cuenta de usuario.
- La aplicación debe disponer de una pantalla en la que los usuarios puedan visualizar todos los datos de su cuenta.
- La aplicación debe permitir a los usuarios agregar listas de medicamentos a su cuenta.
- La aplicación debe permitir a los usuarios agregar medicamentos a cualquiera de sus listas, usando para ello un reconocedor de imágenes.
- La aplicación debe proporcionar información detallada sobre los medicamentos y sus efectos secundarios.
- La aplicación debe permitir a los usuarios editar y eliminar listas de medicamentos, así como también debe permitirles eliminar medicamentos.
- La aplicación debe enviar alertas a los usuarios cuando un medicamento caduque. Dichas alertas deben ser enviadas a todos los dispositivos en los que el usuario tenga iniciada sesión, así como a todos los dispositivos en los que tenga iniciada sesión cualquier usuario al que se le haya compartido la lista en la que se haya el medicamento caducado.
- La aplicación debe disponer de la opción de compartir una lista con otros usuarios registrados.
- La aplicación debe disponer de una pantalla en la que los usuarios puedan visualizar y eliminar todos aquellos medicamentos que estén caducados (tanto de las listas de las que sean propietarios como de aquellas que les hayan compartido).
- La aplicación debe permitir a los usuarios ponerse en contacto con el servicio técnico de la aplicación.

4.2 Requerimientos no Funcionales

- La aplicación debe ser fácil de usar y tener una interfaz intuitiva.
- La aplicación debe ser compatible con el sistema operativo Android.
- La aplicación debe ser accesible desde cualquier dispositivo con conexión a internet.
- La aplicación debe ser segura y proteger la información personal del usuario.
- La aplicación debe ser escalable para poder manejar grandes cantidades de usuarios y datos.

4.3 Requisitos Funcionales

Nombre	Creación de cuenta de usuario
Tipo	Requisito Funcional
Prioridad	Alta
Descripción	La aplicación debe permitir a los usuarios crear una cuenta de usuario
Entrada	Información personal del usuario (nombre, correo electrónico, contraseña…)
Proceso	Validación de información y creación de cuenta en la base de datos
Salidas	Confirmación de creación de cuenta y acceso a la aplicación

Tabla 1: Requisitos Funcionales - Creación de cuenta de usuario

Nombre	Visualización de datos de la cuenta
Tipo	Requisito Funcional
Prioridad	Media
Descripción	La aplicación debe disponer de una pantalla en la que los usuarios puedan visualizar todos los datos de su cuenta
Entrada	Acción del usuario para visualizar su cuenta
Proceso	Obtención de información de la base de datos
Salidas	Visualización de los datos de la cuenta del usuario

Tabla 2: Requisitos Funcionales - Visualización de datos de la cuenta

Nombre	Agregar lista de medicamentos
Tipo	Requisito Funcional
Prioridad	Alta
Descripción	La aplicación debe permitir a los usuarios agregar listas de medicamentos a su cuenta.
Entrada	Nombre de la lista
Proceso	Validación de información y creación de lista en la base de datos
Salidas	Confirmación de creación de lista y visualización de la lista en la aplicación

Tabla 3: Requisitos Funcionales - Agregar lista de medicamentos

Nombre	Agregar medicamento a lista
Tipo	Requisito Funcional
Prioridad	Alta
Descripción	La aplicación debe permitir a los usuarios agregar medicamentos a cualquiera de sus listas, usando para ello un reconocedor de imágenes
Entrada	Imagen del medicamento
Proceso	Reconocimiento de la imagen, validación de información y creación de medicamento en la base de datos
Salidas	Confirmación de creación de medicamento y visualización del medicamento en la lista correspondiente

Tabla 4: Requisitos Funcionales - Agregar medicamento a lista

Nombre	Información detallada de medicamentos
Tipo	Requisito Funcional
Prioridad	Media
Descripción	La aplicación debe proporcionar información detallada sobre los medicamentos y sus efectos secundarios
Entrada	Acción del usuario para visualizar información de un medicamento
Proceso	Obtención de información de la base de datos
Salidas	Visualización de la información del medicamento en la aplicación

Tabla 5: Requisitos Funcionales - Información detallada de medicamentos

Nombre	Editar y eliminar listas y medicamentos
Tipo	Requisito Funcional
Prioridad	Alta
Descripción	La aplicación debe permitir a los usuarios editar y eliminar listas de medicamentos, así como también debe permitirles eliminar medicamentos
Entrada	Acción del usuario para editar o eliminar una lista o medicamento
Proceso	Validación de la acción y actualización de la base de datos
Salidas	Confirmación de la acción y actualización de la lista correspondiente en la aplicación

Tabla 6: Requisitos Funcionales - Editar y eliminar listas y medicamentos

Nombre	**Alertas de medicamentos caducados**
Tipo	Requisito Funcional
Prioridad	Alta
Descripción	La aplicación debe enviar alertas a los usuarios cuando un medicamento caduque. Dichas alertas deben ser enviadas a todos los dispositivos en los que el usuario tenga iniciada sesión, así como a todos los dispositivos en los que tenga iniciada sesión cualquier usuario al que se le haya compartido la lista en la que se haya el medicamento caducado
Entrada	Fecha de caducidad del medicamento
Proceso	Validación de la fecha y envío de alerta a los usuarios correspondientes
Salidas	Envío de alerta a los usuarios correspondientes

Tabla 7: Requisitos Funcionales - Alertas de medicamentos caducados

Nombre	Compartir lista con otros usuarios
Tipo	Requisito Funcional
Prioridad	Media
Descripción	La aplicación debe disponer de la opción de compartir una lista con otros usuarios registrados
Entrada	Acción del usuario para compartir una lista
Proceso	Validación de la acción y actualización de la base de datos
Salidas	Confirmación de la acción y actualización de las listas correspondientes en la aplicación

Tabla 8: Requisitos Funcionales - Compartir lista con otros usuarios

Nombre	Visualización y eliminación de medicamentos caducados
Tipo	Requisito Funcional
Prioridad	Media
Descripción	La aplicación debe disponer de una pantalla en la que los usuarios puedan visualizar y eliminar todos aquellos medicamentos que estén caducados (tanto de las listas de las que sean propietarios como de aquellas que les hayan compartido)
Entrada	Acción del usuario para visualizar o eliminar medicamentos caducados
Proceso	Obtención de información de la base de datos y validación de la acción
Salidas	Visualización o eliminación de los medicamentos caducados correspondientes en la aplicación

Tabla 9: Requisitos Funcionales - Visualización y eliminación de medicamentos caducados

Nombre	Contacto con servicio técnico
Tipo	Requisito Funcional
Prioridad	Baja
Descripción	La aplicación debe permitir a los usuarios ponerse en contacto con el servicio técnico de la aplicación
Entrada	Acción del usuario para contactar con el servicio técnico
Proceso	Redirección a la opción de contacto con el servicio técnico
Salidas	Acceso a la opción de contacto con el servicio técnico

Tabla 10: Requisitos Funcionales - Contacto con servicio técnico

4.4 Requisitos no Funcionales

Nombre	Interfaz fácil de usar
Tipo	Requisito no Funcional
Prioridad	Alta
Descripción	La aplicación debe ser fácil de usar y tener una interfaz intuitiva
Entrada	Acción del usuario para usar la aplicación
Proceso	Diseño de la interfaz
Salidas	Interfaz fácil de usar e intuitiva

Tabla 11: Requisitos no Funcionales - Interfaz fácil de usar

Nombre	Compatibilidad con sistema operativo Android
Tipo	Requisito no Funcional
Prioridad	Alta
Descripción	La aplicación debe ser compatible con el sistema operativo Android
Entrada	Acción del usuario para instalar la aplicación en un dispositivo Android
Proceso	Desarrollo de la aplicación compatible con Android
Salidas	Aplicación compatible con Android

Tabla 12: Requisitos no Funcionales - Compatibilidad con sistema operativo Android

Nombre	Accesible desde cualquier dispositivo con conexión a internet
Tipo	Requisito no Funcional
Prioridad	Alta
Descripción	La aplicación debe ser accesible desde cualquier dispositivo con conexión a internet
Entrada	Acción del usuario para acceder a la aplicación
Proceso	Alojamiento de la aplicación en un servidor accesible desde internet
Salidas	Acceso a la aplicación desde cualquier dispositivo con conexión a internet

Tabla 13: Requisitos no Funcionales - Accesible desde cualquier dispositivo con conexión a internet

Nombre	Seguridad de la información personal del usuario
Tipo	Requisito no Funcional
Prioridad	Alta
Descripción	La aplicación debe ser segura y proteger la información personal del usuario
Entrada	Información personal del usuario (nombre, correo electrónico, contraseña)
Proceso	Implementación de medidas de seguridad para proteger la información del usuario en la base de datos
Salidas	Seguridad de la información personal del usuario

Tabla 14: Requisitos no Funcionales - Seguridad de la información personal del usuario

Nombre	Escalabilidad
Tipo	Requisito no Funcional
Prioridad	Media
Descripción	La aplicación debe ser escalable para poder manejar grandes cantidades de usuarios y datos
Entrada	Incremento de usuarios y datos en la aplicación
Proceso	Diseño y desarrollo de la aplicación para permitir la escalabilidad
Salidas	Capacidad de la aplicación para manejar grandes cantidades de usuarios y datos

Tabla 15: Requisitos no Funcionales - Escalabilidad

4.5 Características de los Usuarios

- Los usuarios son personas que necesitan tomar control de los medicamentos que tienen en casa (tanto de los suyos personales como de familiares).

- Los usuarios pueden ser personas mayores o con discapacidades que pueden tener dificultades para llevar un registro de sus medicamentos.

- Los usuarios pueden ser los hijos o familiares de personas mayores o con discapacidades que necesitan supervisar la toma de medicamentos de sus seres queridos.

- Los usuarios pueden ser enfermeros o cuidadores que trabajan con personas mayores o con discapacidades.

5 Fase de Diseño

Aquí se presenta la fase de diseño del proyecto, cuyo objetivo principal es crear un plan detallado para la implementación del sistema. Durante esta fase se definen los requisitos funcionales y no funcionales del sistema, se diseña la arquitectura y se planifica la implementación.

5.1 Diseño de la Arquitectura

Para el proyecto se utilizó una arquitectura cliente-servidor para el desarrollo del sistema. La aplicación móvil es el cliente y la base de datos de Firebase y los servicios web son los servidores. Se utilizaron las siguientes tecnologías para el desarrollo del sistema:

- Lenguaje de programación: Java
- Sistema operativo: Android
- Base de datos locales: SQLite
- Base de datos online: Firebase Firestore
- Servidor: Servidor de Google empleado por los servicios de Firebase

Finalmente, la arquitectura quedó así:

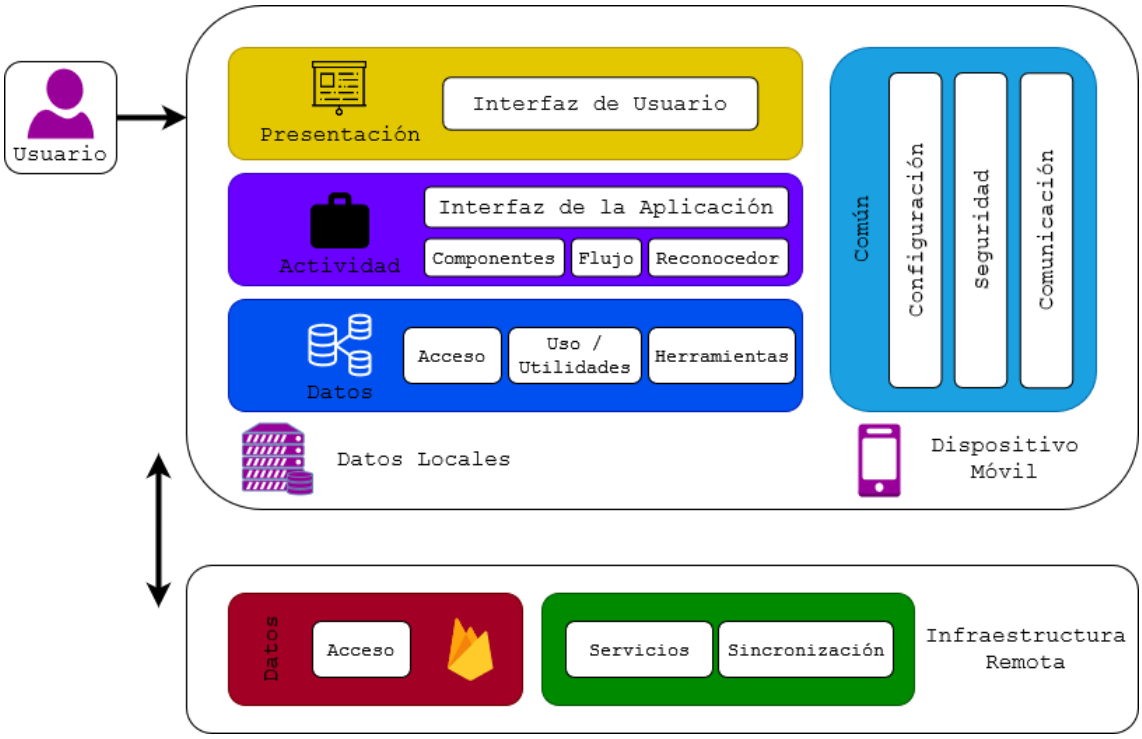

Imagen 1. Arquitectura de la aplicación

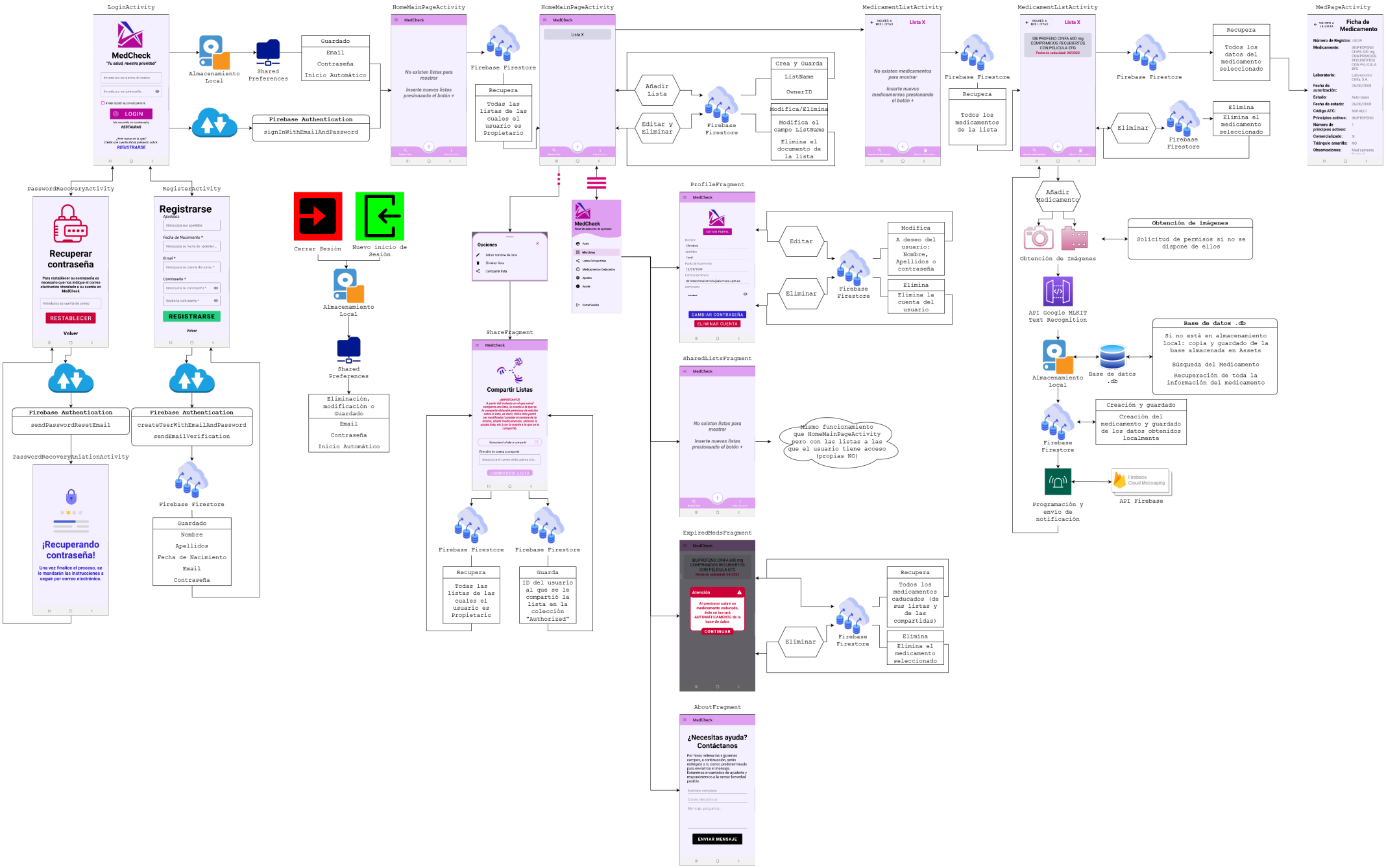

Imagen 2. Arquitectura de la aplicación II

5.2 Diseño de la Interfaz

La interfaz de usuario será diseñada con el objetivo de que sea fácil de usar y proporcione una experiencia de usuario intuitiva. Además, se utilizará un enfoque minimalista para evitar la sobrecarga de información al usuario. La interfaz se diseñará teniendo en cuenta los siguientes puntos:

- Diseño simple y minimalista
- Colores claros y atractivos
- Fuentes legibles y fácil de entender
- Iconos intuitivos y fáciles de entender
- Botones grandes y fáciles de presionar
- Menús desplegables y fáciles de usar

5.3 Evaluación del Diseño

Se llevaron a cabo pruebas de usabilidad, rendimiento e integración para evaluar la eficacia y la eficiencia de la interfaz de usuario. Además, se siguió una metodología de desarrollo ágil y se llevaron a cabo pruebas de funcionamiento y validación durante la fase de pruebas. En la fase de corrección de errores, se realizaron ajustes y correcciones en la aplicación para mejorar su calidad y usabilidad.

5.4 Herramientas

5.4.1 Entorno utilizado: Android Studio

5.4.1.1 Resumen

Android Studio es un entorno de desarrollo integrado (IDE) gratuito y oficial empleado en la creación de aplicaciones móviles que utilicen el sistema operativo Android. Este entorno fue desarrollado por Google y se basa en el popular IDE IntelliJ IDEA. Android Studio es además una herramienta muy potente y completa que ofrece una amplia variedad de características y herramientas para desarrolladores de todos los niveles de experiencia (aunque eso sí, se requieren conocimientos básicos de programación en Java o Kotlin) [3].

Con Android Studio, los desarrolladores pueden crear aplicaciones móviles desde cero o utilizar plantillas predefinidas que faciliten y aceleren el proceso de desarrollo. La plataforma incluye un editor de código avanzado con funciones de autocompletado, resaltado de sintaxis y depuración en tiempo real. Además, también proporciona útiles herramientas de diseño para la interfaz de usuario

como una vista previa en tiempo real de la aplicación en diferentes dispositivos y resoluciones [4].

Una de las características más útiles de Android Studio es su capacidad para integrarse con otras herramientas y servicios de Google (Google Maps, Google Analytics, etc.). Por ejemplo y como posteriormente veremos, los desarrolladores pueden utilizar herramientas como Firebase para gestionar la autenticación de usuarios, el almacenamiento de datos en la nube, y el envío de notificaciones push.

5.4.1.2 Implementación del proyecto

El proceso de desarrollo de este trabajo comenzó con la creación de un nuevo proyecto en Android Studio (seleccionando una plantilla de aplicación vacía). A continuación (y tras es esbozo inicial del prototipado del protecto), se diseñó la interfaz de usuario de la aplicación utilizando el lenguaje XML y empleando la vista previa en tiempo real de la interfaz de usuario, que me permitió ver cómo se vería la aplicación en diferentes dispositivos y resoluciones (aunque para este proyecto me he orientado siguiendo la resolución y diseño de un Samsung Galaxy Note 9).

Una vez que diseñada la interfaz de usuario, comencé a trabajar en la lógica de la aplicación, creando diferentes clases para manejar diferentes aspectos como la gestión de usuarios, la gestión de medicamentos y el envío de notificaciones push. Además, utilicé tecnologías como Firebase Firestore y Firebase Cloud Messaging para almacenar datos y enviar notificaciones; o SQLite para gestionar bases de datos de terceros, como la base de datos de medicamentos proporcionada por la AEMPS (Agencia Española de Medicamentos y Productos Sanitarios) [6].

La aplicación se ha probado en diferentes dispositivos y se corrigieron errores y problemas de rendimiento a medida que se fueron encontrando.

5.4.2 Bases de datos empleadas

5.4.2.1 Base de datos Firebase Firestore

Firebase Firestore es una base de datos en tiempo real en la nube que se utiliza para almacenar y sincronizar datos para aplicaciones móviles, web y otros clientes. Firestore es una base de datos NoSQL que proporciona una estructura de datos flexible y escalable para aplicaciones, además de permitir a los desarrolladores almacenar y sincronizar datos en tiempo real en dispositivos móviles y web, lo que es muy útil para trabajar con datos actualizados en tiempo real sin tener que recargar la aplicación.

En este proyecto, casi todos los datos se almacenan y gestionan empleando esta base de datos (usuarios, listas, expiración de medicamentos, datos sobre cada medicamento añadido a las listas, etc.). Además, Firestore se integró con Firebase Authentication para autenticar a los usuarios y con Firebase Cloud Messaging para enviar el control de las notificaciones [5] [7] [8].

5.4.2.1.1 Estructura de la base de datos

La base de datos desarrollada se compone de dos colecciones: "Lists" y "Users". La colección "Lists" contiene un documento por cada lista, y cada documento tiene dos colecciones (Authorized y Medicaments) y dos campos (listName y ownerUser). La colección "Authorized" tiene un documento por cada usuario autorizado, y cada documento tiene un campo userID que coincide con el ID del documento y el ID del usuario al que se le compartió la lista, obtenido de Firebase Auth. La colección "Medicaments" tiene un documento por cada medicamento, y cada documento tiene varios campos que se rellenarán con los datos obtenidos de la base de datos sql al añadir un nuevo medicamento: "afecta_conduccion", "cod_atc", "comercializado", "estado", "expDate", "fecha_aut", "fecha_estado", "laboratorio", "medicamento", "num_principios_activos", "num_registro", "observaciones", "principios_activos", "problemas_de_suministro", "sustituible" y "triangulo_amarillo".

La colección "Users" contiene un documento por cada usuario (el id de cada documento coincide con el id del usuario, obtenido de Firebase Auth), y cada documento tiene una colección (Tokens) y los campos "birthday", "email", "name", "password" y "surname". La colección "Tokens" contiene un documento por cada dispositivo en el que el usuario ha iniciado sesión, y cada documento tiene un campo tokenID que corresponde al token de cada dispositivo, introducido cuando se inicia sesión y borrado al cerrarla.

```
Lists Collection
├── Document X
│   ├── listName: string
│   ├── ownerUser: string
│   ├── Authorized Collection
│   │   ├── Document X
│   │   │   └── userID: string
│   │   ├── Document Y
│   │   │   └── userID: string
│   │   └── ...
│   └── Medicaments Collection
│       ├── Document X
│       │   ├── afecta_conduccion: string
│       │   ├── cod_atc: string
│       │   ├── comercializado: string
│       │   ├── estado: string
│       │   ├── expDate: string
│       │   ├── fecha_aut: string
│       │   ├── fecha_estado: string
│       │   ├── laboratorio: string
│       │   ├── medicamento: string
│       │   ├── num_principios_activos: int
│       │   ├── num_registro: string
│       │   ├── observaciones: string
│       │   ├── principios_activos: string
│       │   ├── problemas_de_suministro: string
│       │   ├── sustituible: string
│       │   └── triangulo_amarillo: string
│       ├── Document Y
│       │   └── ...
│       └── ...
```

Imagen 3. Estructura de la base de datos Firebase

```
Users Collection
├── Document X
│   ├── birthday: string
│   ├── email: string
│   ├── name: string
│   ├── password: string
│   ├── surname: string
│   └── Tokens Collection
│       ├── Document X
│       │   └── tokenID: string
│       ├── Document Y
│       │   └── tokenID: string
│       └── ...
├── Document Y
│   └── ...
└── ...
```

Imagen 4. Estructura de la base de datos Firebase

5.4.2.2 Base de datos SQL

Esta base de datos fue originalmente un archivo de Excel (descargado del sitio web de la AEMPS), y más concretamente, una versión en línea del Nomenclátor de la Agencia Española de Medicamentos y Productos Sanitarios (AEMPS), que es una lista de medicamentos autorizados en España. Luego, se convirtió a un archivo .sql utilizando una herramienta de conversión de Excel a SQL. Finalmente, el archivo .sql se convirtió a un archivo .db para que pudiera ser utilizado en el proyecto y gestionado por la clase "Database". Esta clase contiene dos métodos principales, "searchMed" y "getMed", que buscan medicamentos en la base de datos y devuelven información sobre ellos.

El método "searchMed" busca medicamentos que contienen una cadena de búsqueda dada en su nombre y devuelve una lista de nombres de medicamentos que coinciden. El método "getMed" busca un medicamento específico por su nombre y devuelve información detallada sobre él, incluyendo su número de registro, laboratorio, fecha de autorización, estado, código ATC, principios activos y más.

Para acceder a la base de datos, la clase "Database" utiliza el método "createDatabase" que comprueba si la base de datos ya existe. Si no existe, la base de datos se crea de forma local en el dispositivo, copiando el archivo .db de la carpeta "assets" de la aplicación en el directorio de la base de datos de la aplicación. La clase "Database" también proporciona un método "close" para cerrar la conexión a la base de datos cuando ya no es necesaria.

5.4.3 Reconocedor de imágenes

5.4.3.1 Procedimiento del reconocedor de imágenes

El proceso de captura, reconocimiento y tratamiento del texto de una imagen en este proyecto comienza con la apertura de la cámara del dispositivo móvil y la captura de una imagen del medicamento y otra de la fecha de caducidad del mismo. Una vez capturadas las imágenes, se procede al reconocimiento del texto en ambas imágenes utilizando la API de ML Kit de Google. Para ello, se utiliza la clase "TextRecognizer" y se configura con "TextRecognizerOptions.DEFAULT_OPTIONS".

El reconocimiento de texto se realiza en segundo plano utilizando el método "process()" de la clase "TextRecognizer". Este método toma una instancia de "InputImage" como parámetro (creada a partir de la imagen capturada anteriormente) y la configura con un objeto "ImageDecoder" para decodificar la imagen capturada y convertirla en un formato que pueda ser procesado por la API de ML Kit.

Una vez que se ha realizado el reconocimiento de texto de ambas imágenes, se obtienen dichos textos y se guardan en formato "String". Estos textos se utilizan para buscar el nombre del medicamento en una base de datos de medicamentos obtenida de la "Agencia Española de Medicamentos y Productos Sanitarios" (AEMPS). Si se encuentra el nombre del medicamento en la base de datos, se procesa la fecha de caducidad para finalmente almacenar todos los datos en la base de datos de Firestore. Si todos los datos se ha guardado correctamente, se programa y envía una notificación (tanto a todos los dispositivos en los que el usuario tenga iniciada sesión, como a todos los dispositivos de las personas a las que se les compartió la lista) utilizando el método "sendMessage()".

5.4.3.2 Limitaciones del reconocedor de imágenes

El proceso de reconocimiento de texto en imágenes puede ser un proceso complejo y que depende de varios factores, como la calidad de la imagen, el tamaño y la posición del texto, el idioma del texto, entre otros. Aunque las soluciones de reconocimiento de texto han mejorado significativamente en los últimos años gracias a la utilización de algoritmos de Machine Learning y redes neuronales, es posible que el reconocimiento de texto no sea del todo preciso y que se necesite realizar una validación manual de los resultados obtenidos.

En este código, se utiliza la API de ML Kit de Google para el reconocimiento de texto en imágenes. Esta API utiliza una red neuronal pre-entrenada para el reconocimiento de texto, lo que permite una mayor precisión y eficacia en la detección del texto. Sin embargo, es importante tener en cuenta que esta solución no es perfecta y puede haber casos en los que el reconocimiento de texto falle.

Una de las razones por las que el reconocimiento de texto puede fallar es porque el método de reconocimiento se realiza de manera limitada, lo que significa que el reconocedor no es todo lo bueno que podría ser debido a la falta de tiempo y recursos para elaborar una red neuronal o IA capaz de reconocer el texto de manera más precisa (como aquellos que usan algoritmos de Machine Learning y redes neuronales), además de que como se indicó previamente, el reconocimiento puede verse afectado por la calidad de la imagen capturada, la iluminación, la posición del texto, entre otros factores.

Es importante tener en cuenta que el reconocimiento de texto en imágenes es un proceso que se está desarrollando y mejorando constantemente, por lo que a medida que se desarrollen y mejoren las tecnologías de Machine Learning y reconocimiento de imagen, es probable que el reconocimiento de texto en imágenes mejore en precisión y eficacia, y con el tiempo y recursos adecuados, podría utilizarse para mejorar esta aplicación.

6 Fase de Desarrollo

6.1 Breve explicación de las clases que componen el proyecto.

6.1.1 MainActivity

La actividad “MainActivity” es la pantalla de carga de la aplicación. Su función es controlar el inicio de sesión automático del usuario y redirigirlo a la pantalla de inicio de la aplicación o a la pantalla de inicio de sesión (según corresponda). En ella se crea un hilo que espera 3.5 segundos antes de continuar con el proceso de inicio de sesión automático (tiempo en el que se muestra la animación de inicio de la aplicación). Después, se recuperan los datos de inicio de sesión guardados en la memoria compartida del dispositivo (SharedPreferences) y se intenta iniciar sesión automáticamente con ellos, empleando la función “signInWithEmailAndPassword()” proporcionada por FirebaseAuth. Si la autenticación es exitosa, se redirige al usuario a la pantalla de inicio de la aplicación.

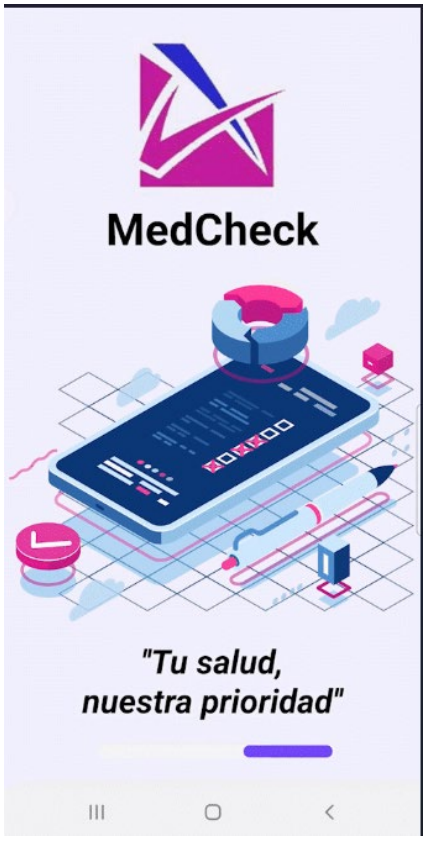

Imagen 5. Pantalla MainActivity

6.1.2 LoginActivity

"LoginActivity" es la pantalla que se muestra cuando el usuario necesita iniciar sesión o registrarse en la aplicación. Su funcionamiento es bastante sencillo. Una vez que el usuario ha introducido los datos de inicio de sesión, la clase recupera los campos de correo electrónico, contraseña, comprueba si el usuario ha seleccionado la opción de "Recordar mi sesión" y salva los datos de inicio de sesión en la memoria compartida del dispositivo (SharedPreferences). Después, se emplea la función "signInWithEmailAndPassword()" proporcionada por FirebaseAuth para iniciar sesión en la cuenta del usuario utilizando su correo electrónico y contraseña. Si la autenticación es exitosa, se redirige al usuario a la pantalla de inicio de la aplicación.

En caso de que se produzca un error en el inicio de sesión, se muestra un mensaje de error al usuario. Si el usuario ha olvidado su contraseña, puede utilizar el botón de recuperación de contraseña para recibir un correo electrónico con las instrucciones para restablecerla.

En el caso de que el usuario no esté registrado, puede utilizar el botón registrarse para ir a la pantalla de Registro y darse de alta en la aplicación.

Cabe destacar que todo el procedimiento explicado anteriormente solo se llevará a cabo si el usuario ha verificado previamente su cuenta. En caso contrario, se mostrará un mensaje solicitándole verificarla.

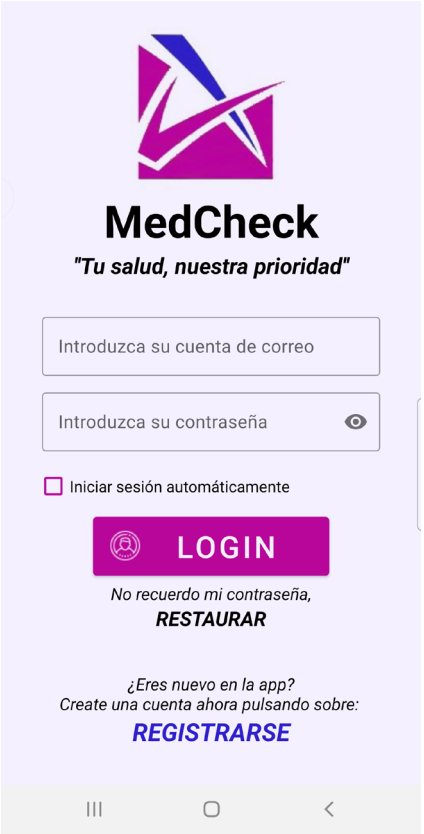

Imagen 6. Pantalla LoginActivity

6.1.3 RegisterActivity

Esta clase es responsable de permitir que los usuarios se registren en la aplicación. Al intentar registrarse, la clase llama a los métodos “birthday_checker()”, “email_checker()” y “password_checker()” para verificar que la información ingresada por el usuario sea válida. Si la información es válida, se utiliza Firebase Authentication para crear una cuenta de usuario y se llama al método “uploadUserToDb()” para cargar la información del usuario en la base de datos de Firebase Firestore.

El método “birthday_checker()” asegura que el usuario tenga al menos 18 años de edad y que la fecha de nacimiento se haya ingresado correctamente; el método “email_checker()” asegura que se haya ingresado una dirección de correo electrónico válida; el método “password_checker()” asegura que la contraseña tenga al menos 8 caracteres, contenga letras, números y caracteres especiales. Si la información no es válida, se muestra un mensaje de error.

Por último, el método “uploadUserToDb()” crea un objeto “User_medCheck” y lo carga en la base de datos de Firebase Firestore. Este objeto contiene información como el nombre, apellidos, fecha de nacimiento, correo electrónico y contraseña del usuario.

Tras el registro en la aplicación, se le envía al usuario un correo para que verifique su intención de registrarse en la aplicación. Si el usuario no verificase su cuenta, no podría iniciar sesión posteriormente.

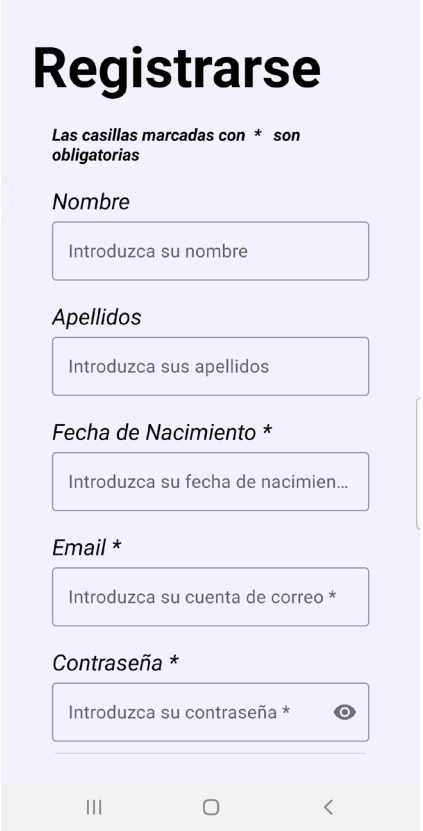

Imagen 7. Pantalla RegisterActivity

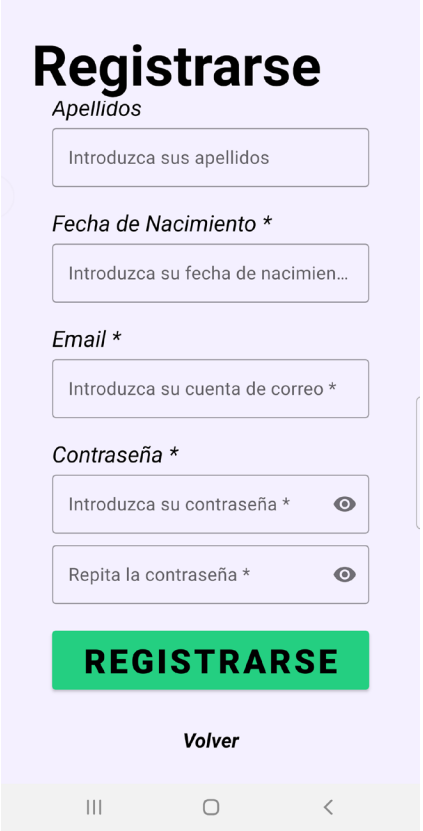

Imagen 8. Pantalla RegisterActivity

6.1.4 PasswordRecoveryActivity y PssdRecoveryAnimationActivity

La clase “PasswordRecoveryActivity” es responsable de manejar la recuperación de contraseñas. Primero, se comprueba que la dirección de correo electrónico sea válida (sea una dirección de correo y además exista una cuenta con dicho correo asignado) y luego se llama al método “sendPasswordResetEmail()” de la instancia de FirebaseAuth para enviar un correo electrónico de recuperación de contraseña al usuario correspondiente. Si la tarea es exitosa, se inicia una instancia de la clase “PasswordRecoveryAnimationActivity” y se muestra un mensaje de éxito.

La clase “PasswordRecoveryAnimationActivity” simplemente muestra una animación de espera durante unos segundos y, a continuación, inicia una instancia de la clase “LoginActivity” y muestra un mensaje de éxito.

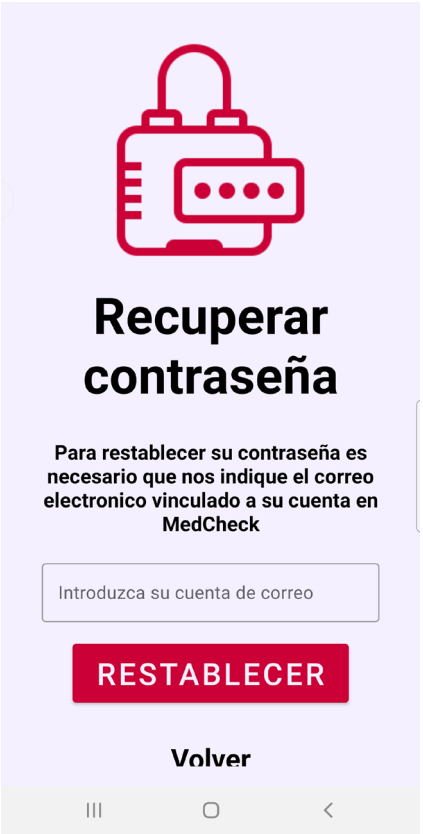

Imagen 9. Pantalla PasswordRecoveryActivity

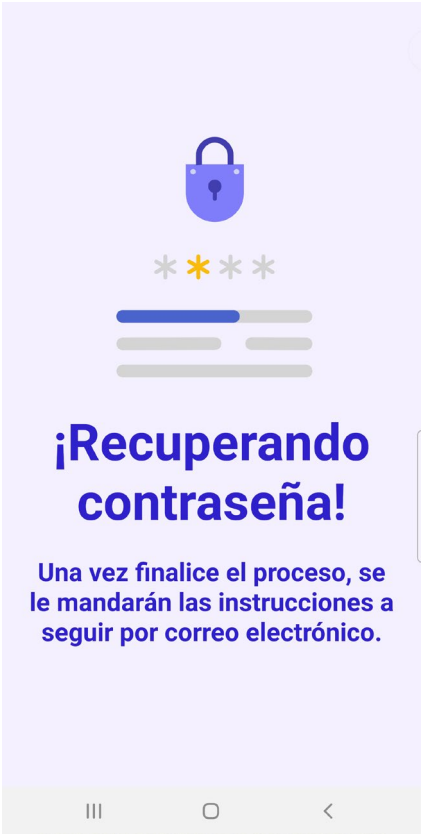

Imagen 10. Pantalla passwordRecoveryAnimationActivity

6.1.5 HomeMainPageActivity

Esta es la actividad principal de la aplicación y se encarga de mostrar las diferentes listas de medicamentos del usuario. Esta consta de una barra de herramientas, una vista de lista, un menú de navegación y un botón flotante para agregar nuevas listas.

Inicialmente, en la función “onCreate()”, se establece la vista de la actividad y se inicializan los elementos de la interfaz de usuario. Además, se configura un listener para la vista de lista que se actualiza cada vez que se agrega o elimina una lista de medicamentos, lo que mantiene al usuario actualizado

constantemente. En este listener se obtienen todas las listas del usuario y se actualiza la vista de lista.

El resto del código está relacionado con la inicialización de la base de datos, la configuración de elementos de la interfaz de usuario y la creación de listeners para eventos específicos. Se destacan como las más importantes:

- La inicialización de la lista de medicamentos en la función onCreate.
- La creación del adapter para la vista de lista.
- La definición del listener para la vista de lista y la obtención de las listas de medicamentos del usuario.
- La definición de la función onItemClick para la vista de lista que permite al usuario seleccionar una lista de medicamentos y acceder a ella.
- La creación del cuadro de diálogo para agregar una nueva lista de medicamentos en la función showAddDialog.
- La definición de la función onNavigationItemSelected para responder a eventos de selección de elementos del menú de navegación.
- La definición de la función showBottomDialog para mostrar un menú emergente con diferentes opciones para el usuario.

Por último, el método "showLogoutAlert" muestra un diálogo de alerta cuando el usuario desea cerrar sesión. Este se utiliza para limpiar las preferencias compartidas y eliminar el token de autenticación del usuario actual, así como cerrar la sesión del usuario dentro de la aplicación.

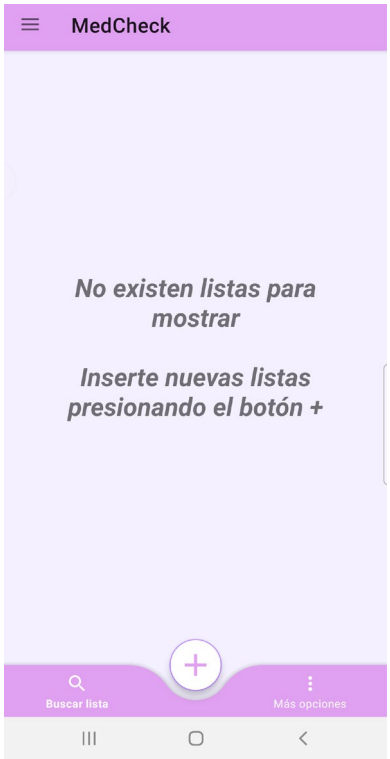

Imagen 11. Pantalla HomeMainPageActivity

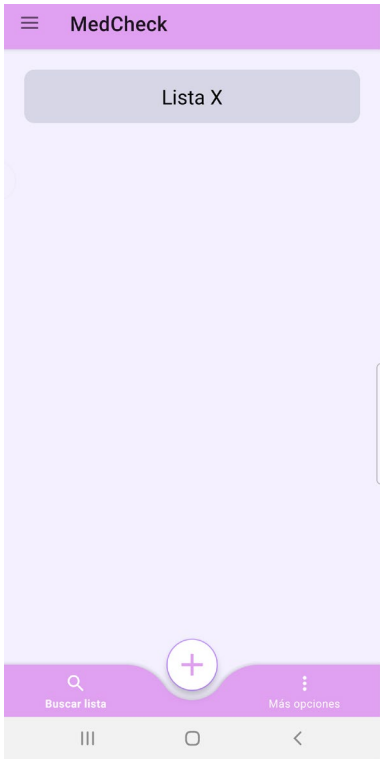

Imagen 12. Pantalla HomeMainPageActivity con listas

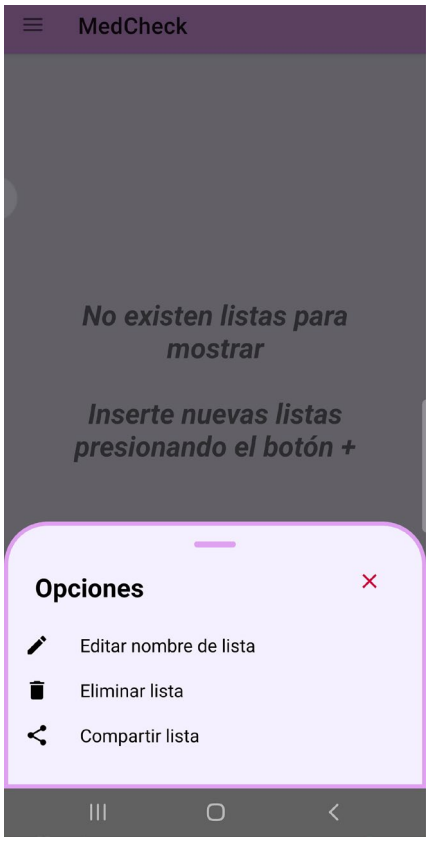

Imagen 13. Pantalla HomeMainPageActivity - Desplegable

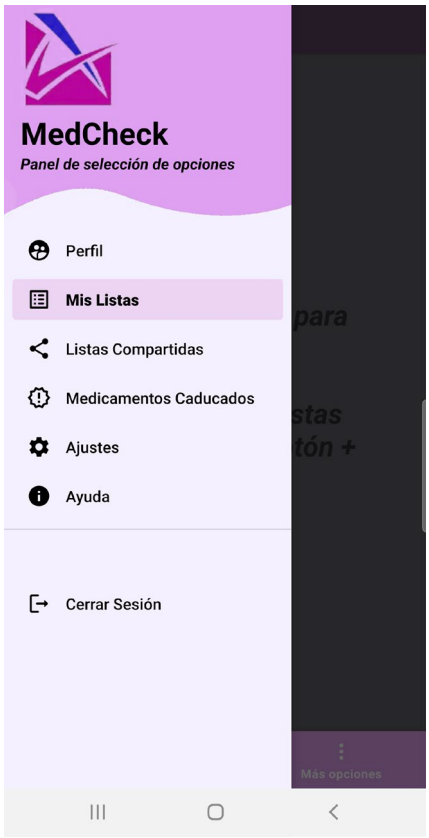

Imagen 14. Pantalla HomeMainPageActivity – Panel

6.1.6 MainPageActivity

La clase "MainPageActivity" se encarga de implementar la interfaz "NavigationView.OnNavigationItemSelectedListener" y es responsable de mostrar el menú principal y los diferentes fragmentos de la aplicación.

Inicialmente se inicializan los diferentes elementos de la interfaz de usuario y se define el comportamiento cuando se selecciona un elemento del menú de navegación. También se recibe un parámetro "option" (proporcionado por la actividad "HomeMainPageActivity" en su llamada/Intent) que se utiliza para determinar qué fragmento se debe mostrar inicialmente.

6.1.7 MedicamentListActivity

La actividad "MedicamentListActivity" es el controlador principal para la lista de medicamentos. Aquí se manejan varias acciones, como agregar medicamentos, buscar medicamentos, eliminar medicamentos y mostrar medicamentos en la lista.

Para agregar un medicamento, el usuario hace clic en el botón flotante en la parte inferior derecha de la pantalla. Al hacer clic en este botón, se abre un cuadro de diálogo que permite al usuario tomar una foto del medicamento y su fecha de caducidad (bien sea empleando la cámara o una foto guardada en la galería). La aplicación luego utiliza el kit de herramientas de aprendizaje automático de Google para reconocer el texto en las imágenes. Si se reconoce el

texto, se realiza una búsqueda en la base de datos para verificar si el medicamento ya existe en la lista. Si el medicamento ya existe, se actualiza la fecha de caducidad. Si el medicamento no existe, se agrega a la lista. La fecha de caducidad del medicamento se almacena en formato de fecha y hora UNIX para permitir la configuración de una alarma en el futuro.

Para eliminar un medicamento, el usuario hace clic en el botón "Eliminar" en la barra de navegación inferior. Selecciona un medicamento de la lista emergente que se le mostrará y posteriormente se elimina de la base de datos.

Todos estos procesos, usan por tanto Firebase Firestore para almacenar y recuperar medicamentos. Los medicamentos se almacenan en la colección "Medicaments" de un documento propio de cada lista en la colección "Lists". Los medicamentos se recuperan utilizando un listener para actualizar la lista en tiempo real.

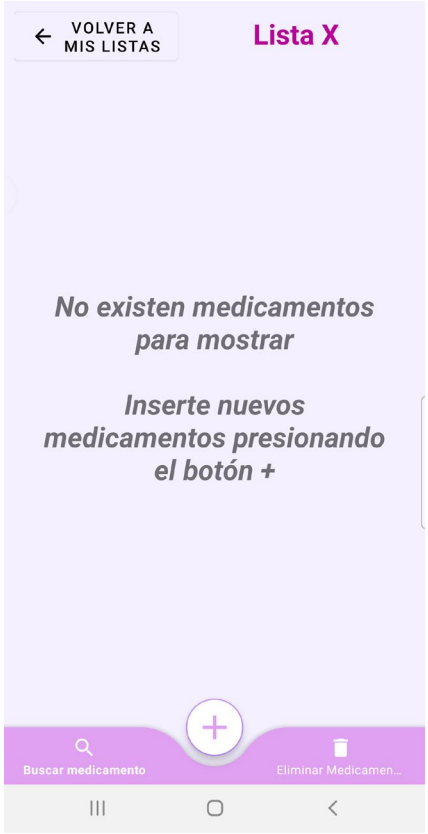

Imagen 15. Pantalla MedicamentListActivity

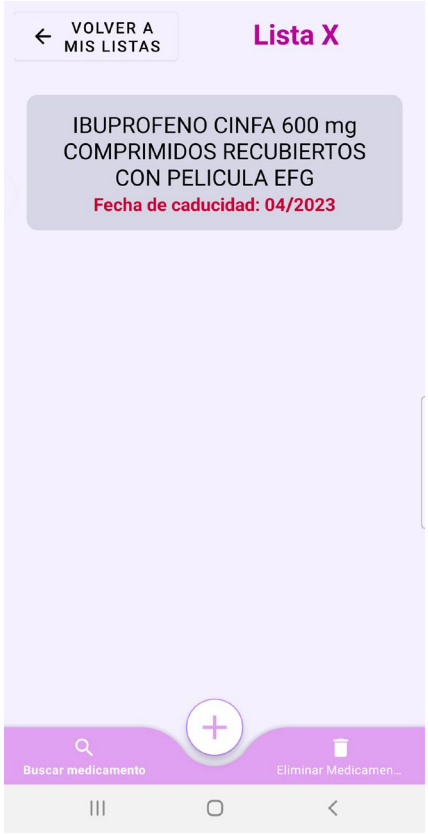

Imagen 16. Pantalla MedicamentListActivity con medicamentos

6.1.8 MedPageActivity

Es la actividad encargada de mostrar la información detallada de un medicamento seleccionado. Para ello, obtener los datos del medicamento (el cual se le pasa como dato en el intent) de la base de datos Firestore y muestra cada atributo del medicamento en su respectivo TextView.

Es importante destacar la forma en que se obtienen los datos del medicamento, utilizando una consulta que filtra los medicamentos por nombre y pertenencia a la lista actual. Además, se utiliza el método “addOnSuccessListener” para

asegurarse de que los datos se obtengan correctamente antes de mostrarlos en la interfaz de usuario.

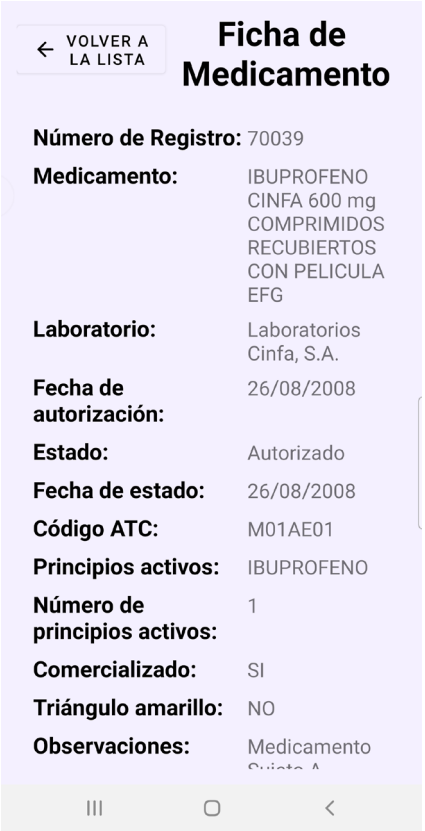

Imagen 17. Pantalla MedPageActivity

6.1.9 ListsFragment

Este fragmento es responsable de mostrar una lista de elementos. La clase extiende de “Fragment” y sobrescribe el método “onCreateView” para inflar el diseño del fragmento. En este caso, se infla el diseño “fragment_lists.xml”. Este diseño contiene un “RecyclerView” que se utiliza para mostrar la lista de elementos.

6.1.10 MedListFragment

Este fragmento es responsable de mostrar una lista de medicamentos. Al igual que el fragmento anterior, este extiende de “Fragment” y sobrescribe el método “onCreateView” para inflar el diseño del fragmento. En este caso, se infla el diseño “fragment_med_list.xml”. Este diseño contiene un “RecyclerView” que se utiliza para mostrar la lista de medicamentos.

En ambos fragmentos, la lógica para mostrar los elementos se encuentra en los adaptadores correspondientes, que no se muestran aquí. En el caso de “MedListFragment”, por ejemplo, el adaptador se encarga de cargar la lista de medicamentos desde la base de datos y mostrarlos en el “RecyclerView”.

6.1.11 ShareListsActivity

La clase “ShareListsActivity” es una actividad que permite al usuario ver todas las listas de medicamentos que otros usuarios le han compartido. La actividad utiliza la base de datos para almacenar y recuperar información sobre las listas y los usuarios autorizados para verlas. En el método “onCreate”, la actividad llama a la base de datos para obtener todas las listas y luego itera sobre ellas para verificar si el usuario actual tiene autorización para ver cada una. Si el usuario está autorizado para ver una lista, se agrega a las listas que se muestran en la pantalla.

El resto de la actividad funciona igual que la actividad “HomeMainPageActivity”, pues al haberle compartido las listas, el usuario recibe permisos completos sobre ellas, pudiendo modificarlas, añadir medicamentos, eliminarlos, etc.

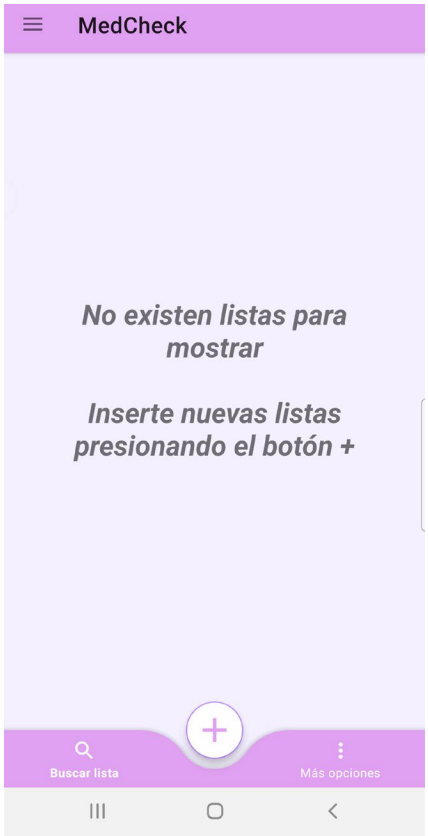

Imagen 18. Pantalla ShareListsFragment

6.1.12 ShareFragment

Esta clase permite al usuario compartir una lista de medicamentos con otro usuario. Lo primero que se hace en esta clase es obtener el ID del usuario actualmente conectado a través de Firebase Authentication. A continuación, se obtiene una lista de todas las listas de medicamentos que el usuario actual ha creado (esto se hace mediante una consulta a la base de datos de Firebase utilizando la instancia de “FirebaseFirestore”). Si se encuentra al menos una lista de medicamentos, se rellena un “Spinner” con los nombres de todas las listas.

Una vez que el usuario ha seleccionado una lista de medicamentos y ha ingresado el correo electrónico del usuario con el que desea compartirla, se realiza otra consulta a la base de datos de Firebase para verificar que el usuario especificado existe y así obtener su ID. Si el correo electrónico es válido y el usuario existe, se llama al método “addUserToAuthorized” que agrega al usuario a la lista de usuarios autorizados para ver la lista de medicamentos seleccionada. Este método también verifica que el usuario no tenga ya acceso a la lista seleccionada.

Imagen 19. Pantalla ShareFragment

6.1.13 ExpiredMedsFragment

Este fragmento se encarga de mostrar los medicamentos caducados del usuario (tanto de una lista de la que es usuario como de una lista compartida). Al iniciarse, se obtiene la instancia actual del mes y año y se llama a la función “searchMeds()”, la cual a su vez llama a la función “searchInAllLists()” que busca en las listas del usuario y de los usuarios autorizados para encontrar medicamentos caducados. Si se encuentran medicamentos, se crea un “ListView” que muestra los nombres y fechas de caducidad de los medicamentos. Si no se encuentra ningún medicamento, se muestra el mensaje "nonExistingMeds".

Además, esta clase cuenta con el método “deleteMed()”, encargado de eliminar un medicamento. Cuando se hace clic en un medicamento en el “ListView”, se llama a esta función. Si el medicamento se elimina correctamente, se llama a “searchMeds()” nuevamente para actualizar la lista de medicamentos.

Imagen 20. Pantalla ExpiredMedsFragment

Imagen 21. Pantalla ExpiredMedsFragment-Aviso

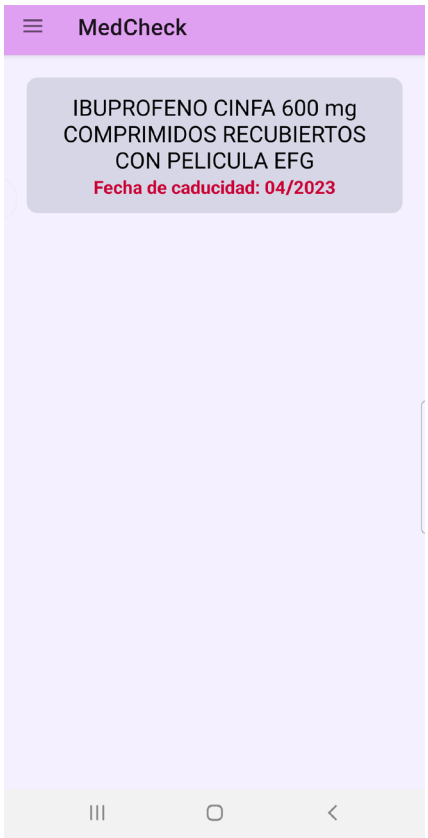

Imagen 22. ExpiredMedsFragment con medicamentos

6.1.14 AboutFragment

Este fragmento se utiliza para mostrar información sobre la aplicación y permitir que los usuarios envíen comentarios a los desarrolladores.

El método “setOnClickListener” establece una acción para el botón de enviar. Cuando se hace clic en el botón, se recupera la información del usuario de los campos de texto y se valida para asegurarse de que se proporcionó información válida. Si los campos son válidos, se crea un “Intent” para enviar un correo electrónico a la dirección de correo electrónico del desarrollador con la información del usuario y el mensaje.

El método “setData” se utiliza para establecer la información del correo electrónico, como la dirección de correo electrónico del destinatario, el asunto y el mensaje. Posteriormente, se llama al método “startActivity” para iniciar la aplicación de correo electrónico predeterminada del dispositivo con la información de correo electrónico.

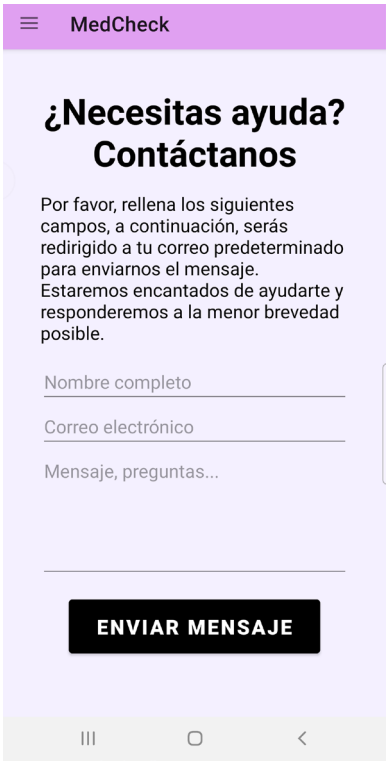

Imagen 23. ShareFragment

6.1.15 ProfileFragment

Esta clase es de las más simples del proyecto, pues se encarga de mostrar y editar el perfil de usuario, así como de permitir al usuario cambiar su contraseña.

Imagen 24. ProfileFragment

6.1.16 FirebaseMessagingService_medCheck

La clase "FirebaseMessagingService_medCheck" se utiliza para recibir notificaciones y programar alarmas en la aplicación. Al recibir una nueva notificación, se crea un intent para mostrar la notificación en el dispositivo. Se utiliza un objeto "RemoteMessage" para acceder a la información de la notificación, como su título y cuerpo. Luego, se programa una alarma utilizando un objeto "AlarmManager" y se establece el tiempo de la alarma utilizando el valor de fecha que se recibe en la notificación.

La clase también implementa el método "onNewToken", que se llama cuando se genera un nuevo token de Firebase Cloud Messaging (FCM). Se utiliza un objeto "SharedPreferences" para almacenar el token y poder acceder a él más tarde.

Es importante destacar que la clase depende de la existencia de una clase llamada "NotificationReceiver_medCheck", que es responsable de mostrar la notificación en el dispositivo.

6.1.17 NotificationReceiver_medCheck

Esta es una subclase de “BroadcastReceiver”. Cuando se recibe una notificación, se reproduce un sonido y vibra. Luego, se extraen los datos de la notificación y se crea una notificación personalizada usando la clase “NotificationCompat.Builder”. Se establece el título, el texto y el estilo de la notificación, así como también se crea un “PendingIntent” que se activará cuando se haga clic en la notificación. Finalmente, se crea un “NotificationManager” y se llama al método “notify()” para mostrar la notificación.

Cabe mencionar que se establece un canal de notificación si el dispositivo está ejecutando la versión de Android Oreo o superior. Esto se hace utilizando la clase “NotificationChannel”.

6.1.18 NotificationSender_medCheck

La clase “NotificationSender_medCheck” se utiliza para enviar notificaciones push a través de Firebase Cloud Messaging (FCM). El constructor de la clase toma como parámetros el token FCM del usuario, el título y el cuerpo de la notificación, el contexto y la actividad desde donde se está enviando la notificación y la fecha asociada a la notificación.

La variable “requestQueue” se inicializa utilizando la librería “Volley” y se crea un objeto “JSONObject” que contiene el token FCM del usuario, el título y el cuerpo de la notificación. Se crea un objeto adicional que contiene la fecha asociada a la notificación. A continuación, se crea un objeto “JsonObjectRequest” que utiliza el método “POST” para enviar la notificación al servidor FCM. Si la notificación se envía correctamente, la respuesta del servidor se maneja en el método “onResponse”. Si hay un error al enviar la notificación, el error se maneja en el método “onErrorResponse”.

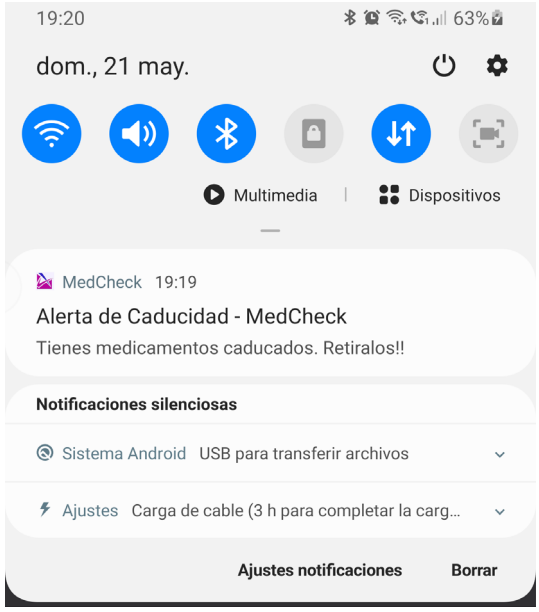

Imagen 25. Notificación MedCheck

7 Pruebas de funcionamiento

Una vez finalizado el desarrollo de la aplicación, debí comprobar si la aplicación funcionaba según lo esperado, por lo que fue fundamental realizar una serie de pruebas. En este apartado se describen las diferentes pruebas que se han llevado a cabo para comprobar el correcto funcionamiento de la aplicación. Estas pruebas se han dividido en tres categorías: pruebas de usabilidad, pruebas de rendimiento y pruebas de integración. Para ello se empleará un formato específico, siguiendo el siguiente ejemplo:

CASO DE PRUEBA	
PROPÓSITO	
PREREQUERIMIENTOS	
DATOS DE ENTRADA	
PASOS	1. 2. . . .
RESULTADO	

7.1 Pruebas de usabilidad

Las pruebas de usabilidad se realizan para evaluar qué tan fácil es usar la aplicación. Estas pruebas pueden evaluar la facilidad de uso, la eficiencia, la satisfacción del usuario y la capacidad de aprendizaje de la aplicación.

7.1.1 Prueba 1: Registro de usuarios

CASO DE PRUEBA	Registro de Usuarios
PROPÓSITO	Comprobar que un nuevo usuario puede registrarse correctamente en la aplicación
PREREQUERIMIENTOS	Ninguno
DATOS DE ENTRADA	Nombre de usuario, fecha de nacimiento, correo electrónico y contraseña
PASOS	1. Ingresar los datos del nuevo usuario en el formulario de registro 2. Comprobar que los datos ingresados son correctos 3. Finalizar el registro y verificar que el usuario es redirigido a la pantalla de inicio de sesión
RESULTADO	El registro se realizó correctamente, comprobándose con éxito que la contraseña, el correo electrónico y la fecha de nacimiento fueran correctos. Tras finalizar el registro, se redirigió al usuario a la pantalla de inicio de sesión.

7.1.2 Prueba 2: Inicio de sesión y verificación de la cuenta

CASO DE PRUEBA	Inicio de sesión y verificación de la cuenta
PROPÓSITO	Comprobar que un usuario registrado puede iniciar sesión y verificar su cuenta correctamente en la aplicación
PREREQUERIMIENTOS	Un usuario registrado y una cuenta de correo electrónico para la verificación de la cuenta
DATOS DE ENTRADA	Correo electrónico y contraseña
PASOS	1. Ingresar el correo electrónico y la contraseña del usuario registrado en el formulario de inicio de sesión 2. Comprobar que el usuario NO ha verificado su cuenta mediante el enlace enviado a su correo electrónico 3. Verificar la cuenta 4. Comprobar que la aplicación permite y realiza correctamente el inicio de sesión una vez verificada la cuenta, redirigiendo al usuario a la pantalla principal de la aplicación.
RESULTADO	Se comprobó como la aplicación no permitió el inicio de sesión mientras que el usuario no hubiera verificado su cuenta mediante el enlace enviado a su correo electrónico. Una vez verificada la cuenta, la aplicación permitió y realizó correctamente el inicio de sesión, redirigiendo al usuario tras el inicio a la pantalla principal de la aplicación.

7.1.3 Prueba 3: Añadir, buscar, modificar y eliminar listas

CASO DE PRUEBA	Añadir, buscar, modificar y eliminar listas de medicamentos
PROPÓSITO	Comprobar que se pueden añadir, buscar, modificar y eliminar listas correctamente
PREREQUERIMIENTOS	Ninguno
DATOS DE ENTRADA	Nombres de las nuevas listas creadas
PASOS	1. Añadir varias listas de medicamentos 2. Realizar una búsqueda entre ellas para comprobar el correcto funcionamiento del buscador 3. Modificar el nombre de una de las listas 4. Eliminar una de las listas.
RESULTADO	Las listas se añaden, buscan, modifican y eliminan correctamente. Tanto el funcionamiento del algoritmo de búsqueda como el del adaptador de listas es el esperado. Además, se comprueba como la aplicación es completamente capaz de obtener datos de la base de datos de Firestore así como también es capaz de mantenerlos actualizados constantemente.

7.1.3.1 Comprobaciones extra derivadas de la prueba 3

7.1.3.1.1 Recuperación, modificación y eliminación de datos en la base de datos Firebase

Como resultados extra de la prueba 3, se comprueba como la aplicación es completamente capaz de obtener, modificar y eliminar datos de la base de datos de Firestore así como también es capaz de mantenerlos actualizados constantemente.

7.1.4 Prueba 4: Añadir medicamento

CASO DE PRUEBA	Añadir medicamento
PROPÓSITO	Comprobar que se puede añadir un nuevo medicamento a una lista (tanto usando la cámara como seleccionando la imagen directamente desde la galería)
PREREQUERIMIENTOS	Una lista de medicamentos creada previamente
DATOS DE ENTRADA	Fotografía del medicamento y fotografía de la fecha de caducidad
PASOS	1. Tomar y añadir fotografía del medicamento 2. Tomar y añadir fotografía de la fecha de caducidad 3. Añadir el nuevo medicamento. 1. Seleccionar y añadir fotografía del medicamento 2. Seleccionar y añadir fotografía de la fecha de caducidad 3. Añadir el nuevo medicamento.
RESULTADO	El medicamento se añadió correctamente a la lista y todos sus datos se obtuvieron (de la base de datos local .db) y guardaron en la base de datos de Firebase. Además, se comprobó como la aplicación es capaz de programar y enviar una notificación (que se mostrará el día en el que el medicamento venza) a todos los dispositivos del usuario en cuestión, así como también a todos los dispositivos de los usuarios a los que se les haya compartido la lista.

7.1.4.1 Comprobaciones extra derivadas de la prueba 4

7.1.4.1.1 Creación, manipulación y uso de la base de datos .db

Como claramente se indica en la “Prueba 4”, tras añadir el medicamento, se confirma que:

- La aplicación reconoce la base de datos .db (ubicada en Assets).
- La aplicación reconoce que no está en la memoria local del dispositivo y crea una copia en dicha memoria local.

- El programa recorre y realiza sucesivas búsquedas dentro de la base de datos para contrastar si el medicamento existe.
- Tras encontrar el medicamento en cuestión, se recuperan y almacenan todos sus datos

7.1.4.1.2 Creación, envío y tratamiento de notificaciones

Tras la "Prueba 4", después de añadir el nuevo medicamento, se comprueba que:

- La aplicación programa y envía una notificación (que se mostrará el día en el que el medicamento venza) a todos los dispositivos del usuario y a todos los dispositivos de los usuarios que tengan acceso a la lista

7.1.5 Prueba 5: Ver información de un medicamento

CASO DE PRUEBA	Ver información de un medicamento
PROPÓSITO	Comprobar que se puede ver la información de un medicamento pulsando sobre él en la lista
PREREQUERIMIENTOS	Un medicamento añadido a una lista de medicamentos
DATOS DE ENTRADA	Ninguno
PASOS	1. Pulsar sobre el medicamento
RESULTADO	Al presionar sobre el medicamento se vieron todos los datos relacionados con él.

7.1.6 Prueba 6: Compartir lista de medicamentos

CASO DE PRUEBA	Compartir lista de medicamentos
PROPÓSITO	Comprobar que se puede compartir una lista de medicamentos con otro usuario registrado en la aplicación
PREREQUERIMIENTOS	Una lista de medicamentos creada previamente y un usuario registrado en la aplicación con el que compartir la lista
DATOS DE ENTRADA	Correo electrónico del usuario con el que se quiere compartir la lista de medicamentos
PASOS	1. Seleccionar la lista de entre todas las listas en el desplegable 2. Introducir el correo electrónico del usuario con el que se quiere compartir 3. Presionar el botón compartir
RESULTADO	La lista de medicamentos se compartió correctamente con el usuario y este tuvo acceso total a dicha lista.

7.1.7 Prueba 7: Gestión de errores

CASO DE PRUEBA	Gestión de errores
PROPÓSITO	Comprobar que la aplicación muestra mensajes de error claros y concisos al usuario en caso de datos incorrectos o incompletos en los formularios
PREREQUERIMIENTOS	Ninguno
DATOS DE ENTRADA	Datos incorrectos o incompletos en los formularios
PASOS	1. Rellenar un formulario (registro, login, etc.) con datos incorrectos o incompletos 2. Intentar enviar el formulario 3. Observar el mensaje de error que muestra la aplicación
RESULTADO	La aplicación mostró mensajes de error claros y concisos para indicar al usuario cuál era problema y cómo solucionarlo

7.2 Pruebas de rendimiento

Las pruebas de rendimiento se realizan para evaluar qué tan bien funciona la aplicación en términos de velocidad, capacidad de respuesta y estabilidad bajo diferentes condiciones de carga. Estas pruebas pueden evaluar la velocidad de carga de la aplicación, la respuesta a las solicitudes de los usuarios y la capacidad de la aplicación para manejar grandes cantidades de datos.

7.2.1 Prueba 1: Prueba de carga

CASO DE PRUEBA	Prueba de carga
PROPÓSITO	Comprobar el rendimiento de la aplicación con una lista de 100 medicamentos ***Fue necesario modificar temporalmente el código de introducción de un nuevo medicamento***
PREREQUERIMIENTOS	Ninguno
DATOS DE ENTRADA	Lista de 100 medicamentos generada automáticamente al añadir un medicamento
PASOS	1. Modificar el código para que añada aleatoriamente 100 medicamentos cuando se intente añadir uno 2. Añadir un medicamento nuevo
RESULTADO	La aplicación respondió correctamente y sin retrasos durante la gestión de la lista de medicamentos

7.3 Pruebas de integración

Las pruebas de integración se realizan para evaluar cómo se integra la aplicación con otros sistemas o componentes. Estas pruebas pueden evaluar cómo funciona la aplicación cuando se conecta a otros sistemas, cómo se manejan los errores de integración y cómo se garantiza la seguridad de los datos al integrar diferentes sistemas.

7.3.1 Prueba 1: Sincronización con la nube

CASO DE PRUEBA	Sincronización con la nube
PROPÓSITO	Comprobar que la aplicación se sincroniza correctamente con la nube y se actualiza en todos los dispositivos en los que el usuario tiene instalada la aplicación
PREREQUERIMIENTOS	Conexión a internet
DATOS DE ENTRADA	Ninguno
PASOS	1. Realizar varias y diferentes pruebas sobre partes del código cuya función es la de recuperar y utilizar datos de la base de datos, manteniendo dichos datos en constante sincronización con la aplicación 2. Comprobar cómo responde la aplicación
RESULTADO	La aplicación se sincronizó correctamente con la nube y se actualizó en todos los dispositivos en los que el usuario tenía instalada la aplicación

7.3.2 Prueba 2: Prueba de integración

CASO DE PRUEBA	Prueba de integración
PROPÓSITO	Comprobar que la aplicación funciona correctamente mientras se realiza una llamada telefónica
PREREQUERIMIENTOS	Línea de teléfono y dos dispositivos móviles distintos
DATOS DE ENTRADA	Llamada entrante o saliente
PASOS	1. Realizar una llamada telefónica mientras se utiliza la aplicación. 2. Comprobar cómo responde la aplicación
RESULTADO	La aplicación se minimizó correctamente durante la llamada y se restauró a su estado anterior una vez finalizada la misma

7.4 Pruebas del reconocedor de imágenes

Como bien se ha comentado previamente, el reconocedor de imágenes utiliza técnicas de procesamiento de imágenes y aprendizaje automático para reconocer el nombre del medicamento y la fecha de caducidad a partir de las fotografías seleccionadas. El éxito del reconocimiento depende en gran medida de la calidad de la foto y de la claridad de la etiqueta del medicamento. Si la etiqueta está borrosa o parcialmente cubierta, el reconocedor de imágenes no será capaz de reconocer el nombre del medicamento ni la fecha de caducidad. Además, si la etiqueta está en otro idioma, el reconocedor de imágenes tampoco será capaz de reconocer la información correctamente.

7.4.1 Prueba 1: Fotografías borrosas

CASO DE PRUEBA	Fotografías borrosas
PROPÓSITO	Comprobar que el reconocedor de imágenes no es capaz de reconocer el nombre del medicamento ni la fecha de caducidad si la fotografía está borrosa
PREREQUERIMIENTOS	Cámara o acceso a la galería
DATOS DE ENTRADA	Fotografías de caja de medicamento y fecha de caducidad borrosas
PASOS	1. Capturar o seleccionar fotografías de una caja de medicamento y/o fecha de caducidad que estén borrosas 2. Intentar añadir el medicamento para comprobar cómo funciona el reconocedor
RESULTADO	El reconocedor de imágenes no es capaz de reconocer el nombre del medicamento

7.4.2 Prueba 2: Fotografías en otro idioma

CASO DE PRUEBA	Fotografías en otro idioma
PROPÓSITO	Comprobar que el reconocedor de imágenes no es capaz de reconocer el nombre del medicamento si la foto es de un medicamento en otro idioma
PREREQUERIMIENTOS	Cámara o acceso a la galería
DATOS DE ENTRADA	Fotografía de una caja de medicamento en otro idioma
PASOS	1. Capturar o seleccionar fotografías de una caja de medicamento en otro idioma 2. Intentar añadir el medicamento para comprobar cómo funciona el reconocedor
RESULTADO	El reconocedor de imágenes no es capaz de reconocer el nombre del medicamento.

7.4.3 Prueba 3: Fotografías claras

CASO DE PRUEBA	Fotografías claras
PROPÓSITO	Comprobar que el reconocedor de imágenes es capaz de reconocer el nombre del medicamento y la fecha de caducidad si las fotografías están claras
PREREQUERIMIENTOS	Cámara o acceso a la galería
DATOS DE ENTRADA	Fotografías de caja de medicamento y fecha de caducidad que se vean claramente
PASOS	1. Capturar o seleccionar fotografías de una caja de medicamento y/o fecha de caducidad que estén claras 2. Intentar añadir el medicamento para comprobar cómo funciona el reconocedor
RESULTADO	El reconocedor de imágenes es capaz de reconocer el nombre y la fecha de caducidad correctamente

7.4.4 Prueba 4: Fotografías parcialmente cubiertas

CASO DE PRUEBA	Fotografías parcialmente cubiertas
PROPÓSITO	Comprobar que el reconocedor no es capaz de reconocer el nombre y la fecha de caducidad si las fotografías están parcialmente cubiertas
PREREQUERIMIENTOS	Cámara o acceso a la galería
DATOS DE ENTRADA	Fotografías de caja de medicamento y fecha de caducidad que se vean parcialmente cubiertas
PASOS	1. Capturar o seleccionar fotografías de una caja de medicamento y/o fecha de caducidad que estén parcialmente cubiertas 2. Intentar añadir el medicamento para comprobar cómo funciona el reconocedor
RESULTADO	El reconocedor de imágenes no es capaz de reconocer el nombre ni la fecha debido a la foto

7.5 Análisis de los resultados

Durante la realización de las pruebas se detectaron algunos errores menores que se fueron corrigiendo de forma inmediata. En general, la aplicación ha respondió de forma correcta y se ha comportó según lo esperado en todas las pruebas realizadas.

Se comprobó que la aplicación es fácil de usar y tiene una interfaz amigable para el usuario. Además, se ha comprobado que la aplicación es capaz de trabajar con grandes cantidades de datos y que responde correctamente en situaciones de alta carga. En cuanto a la sincronización con la nube, se ha comprobado que la aplicación es capaz de sincronizar los datos del usuario de forma rápida y eficiente. Además, se comprobó como la aplicación solo es capaz de reconocer fotografías que se vean claras.

En definitiva, se considera que la aplicación cumple con todos los estándares de calidad necesarios para garantizar una experiencia satisfactoria al usuario.

8 Posibles planes futuros para la aplicación

8.1 Integración con asistentes virtuales

otra posible mejora sería la integración con asistentes virtuales, como Amazon Alexa o Google Assistant. De esta manera, los usuarios podrían interactuar con la aplicación mediante comandos de voz, lo que podría resultar especialmente útil para personas mayores o con discapacidades. Además, los usuarios podrían recibir notificaciones y alertas directamente de su asistente, sin necesidad de tener que sacar el teléfono del bolsillo o bolso.

8.2 Mejoras en el reconocimiento de imágenes

Una posible mejora sería la incorporación de técnicas de aprendizaje automático para mejorar el reconocimiento de imágenes (hablando del campo de reconocimiento de imágenes, los algoritmos de aprendizaje automático han demostrado ser muy efectivos). De esta manera, se podrían detectar medicamentos y fechas de caducidad con mayor precisión y fiabilidad.

Por ejemplo, se podría utilizar un modelo de red neuronal convolucional (CNN) para el reconocimiento de medicamentos a partir de imágenes. Este modelo podría entrenarse con una gran cantidad de imágenes de medicamentos y sus etiquetas correspondientes. Una vez entrenado, el modelo podría utilizarse para identificar medicamentos en nuevas imágenes.

Otra posible técnica de aprendizaje automático que se podría utilizar es el procesamiento del lenguaje natural (NLP) para extraer información tanto del medicamento como de la fecha de caducidad. Esto podría lograrse mediante la identificación de patrones en el texto.

8.3 Integración con farmacias

Otra posible mejora sería la integración con farmacias locales, de manera que los usuarios pudieran recibir alertas e información sobre la disponibilidad de medicamentos en tiempo real: la aplicación podría conectarse con el sistema de inventario de las farmacias para mostrar a los usuarios si el medicamento que les ha caducado está disponible en una farmacia cercana.

9 Informe de impacto en la sociedad

La aplicación desarrollada tiene un gran potencial para mejorar la calidad de vida de las personas mayores o con discapacidades, que pueden tener problemas para llevar un registro de sus medicamentos y sus fechas de caducidad. Al facilitar la gestión de los medicamentos, la aplicación puede contribuir a mejorar la autonomía y la calidad de vida de estas personas.

Un punto importante a destacar es que la aplicación también puede ser útil para los hijos o familiares de personas mayores o con discapacidades dado que la aplicación permite la gestión de los medicamentos y la alerta de caducidad de forma remota, por lo que los hijos o familiares pueden supervisar la caducidad de los medicamentos de sus seres queridos sin necesidad de estar presentes físicamente.

Por otro lado, la aplicación también puede ser de gran ayuda para los enfermeros y cuidadores que trabajan con personas mayores o con discapacidades. Con pequeñas mejoras, la aplicación podría adaptarse para ser empleada en centros de mayores para garantizar una gestión eficiente de los medicamentos y la alerta de caducidad de los mismos, lo que puede contribuir a reducir el riesgo de errores y mejorar la calidad del cuidado.

Además, la aplicación puede tener un impacto positivo en el sistema de salud en general. Al prevenir errores en la toma de medicamentos caducados, la aplicación puede contribuir a reducir los costos asociados a las hospitalizaciones y los tratamientos médicos innecesarios.

La aplicación también puede ofrecer información detallada sobre los medicamentos y sus efectos secundarios, lo que puede ser útil para las personas que necesitan tomar medicamentos de forma regular pero no están familiarizadas con los mismos.

10 Bibliografía

1.- CareZone. (s.f.). 27 de mayo de 2023. https://carezone.com/es/

2.- DailyMed. (s.f.). 27 de mayo de 2023. https://dailymed.nlm.nih.gov/dailymed/

3.- Android Studio. (2021). Android Developers. 27 de mayo de 2023. https://developer.android.com/studio

4.- Getting started with Android development. (2021). 27 de mayo de 2023. https://developer.android.com/guide

5.- Firebase. (2021). Google Developers. 27 de mayo de 2023. https://firebase.google.com/

6.- Android Studio Tutorial For Beginners. (2021). Javatpoint. 27 de mayo de 2023. https://www.javatpoint.com/android-studio-tutorial-for-beginners

7.- Documentación oficial de Firebase. 27 de mayo de 2021. https://firebase.google.com/docs/firestore

8.- Guía de Firebase Firestore de Android Developers. https://developer.android.com/guide/firestore

Este documento esta firmado por

	Firmante	CN=tfgm.fi.upm.es, OU=CCFI, O=ETS Ingenieros Informaticos - UPM, C=ES
	Fecha/Hora	Tue May 30 16:32:55 CEST 2023
	Emisor del Certificado	EMAILADDRESS=camanager@etsiinf.upm.es, CN=CA ETS Ingenieros Informaticos, O=ETS Ingenieros Informaticos - UPM, C=ES
	Numero de Serie	561
	Metodo	urn:adobe.com:Adobe.PPKLite:adbe.pkcs7.sha1 (Adobe Signature)